Assignment-13.1

Perumalla Sushwanth

batch 29

2303a51567

Task 1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
  - Prompt AI to identify duplicate logic and replace it with functions or classes.
  - Ensure the refactored code maintains the same output.
  - Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
  - Refactored code with a reusable function and no duplication.
  - Well documented code

CODE:

```
ai_assistedcoding > lab_13.1 > ...
# Refactor the given Python script to remove duplicate rectangle calculations by creating reusable functions with proper docstrings, ensuring the output remains unchanged
#!/usr/bin/env python3
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.
    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    int or float: The calculated area of the rectangle.
    """
    return length * width
def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.
    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    int or float: The calculated perimeter of the rectangle.
    """
    return 2 * (length + width)
def print_rectangle_details(length, width):
    """
    Print the area and perimeter of a rectangle.
    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.
    """
    print("Area of Rectangle:", calculate_area(length, width))
    print("Perimeter of Rectangle:", calculate_perimeter(length, width))
# Rectangle Data
rectangles = [
    (5, 10),
    (7, 12),
    (10, 15)
]
# Process each rectangle
for length, width in rectangles:
    print_rectangle_details(length, width)
```

OUTPUT:

```
AI_ASS_CODING'; & 'C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\perum\.vscode\extensions\ms-python.debugpperum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1'
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Instructions:
  - Identify repeated or related logic and extract it into reusable functions.
  - Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- Expected Output:
  - Code with a function calculate_total(price) that can be reused for multiple price inputs.
  - Well documented code

CODE:

```python
# Refactor the legacy Python script by extracting the repeated tax calculation logic into a reusable function calculate_total(price) with proper docstrings while maintaining the same output
def calculate_total(price):
    """
    Calculate the total price including 18% tax.

    Parameters:
    price (int or float): The original price of the product.

    Returns:
    float: Total price after adding 18% tax.
    """
    tax = price * 0.18
    total = price + tax
    return total


# Prices to process
prices = [250, 500]

# Print total price for each item
for price in prices:
    print("Total Price:", calculate_total(price))
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  c:; cd 'c:\Users\perum\OneDrive\Desktop\AI_ASS_CODING'; & 'C:\Users\perum\AppData\Local\Pro
ng\lab_13.1'
Total Price: 295.0
Total Price: 590.0
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:

    - Accept marks as a parameter.
    - Return the corresponding grade as a string.
    - The grading logic must strictly follow the conditions below:
- Marks ≥ 90 and ≤ 100 → "Grade A"
- Marks ≥ 80 → "Grade B"
- Marks ≥ 70 → "Grade C"
- Marks ≥ 40 → "Grade D"
- Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

1. Include proper docstrings for:
    - The class
    - The method (with parameter and return descriptions)
2. The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

marks = 85

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

marks = 72

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

CODE:

```python
# Refactor the repeated grading conditional logic into an object-oriented Python solution using a class GradeCalculator
# with a reusable method calculate_grade(self, marks) including proper docstrings
# while preserving the exact grading conditions.
class GradeCalculator:
    """
    A class used to calculate student grades based on marks.

    Methods
    -------
    calculate_grade(self, marks):
        Returns the grade corresponding to the given marks.
    """
    def calculate_grade(self, marks):
        """
        Determine the grade based on the student's marks.

        Parameters:
        marks (int or float): The marks obtained by the student (0-100).

        Returns:
        str: The grade corresponding to the marks.
        """
        if 90 <= marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        elif marks >= 0:
            return "Fail"
# Create object of GradeCalculator
grade_calculator = GradeCalculator()

# Call method for different students
marks_list = [85, 72, 95, 38]

for marks in marks_list:
    print(f"Marks: {marks} → {grade_calculator.calculate_grade(marks)}")
```

OUTPUT:

```
c:\Users\perum\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63352' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcodi
ng\lab_13.1'
Marks: 85 → Grade B
Marks: 72 → Grade C
Marks: 95 → Grade A
Marks: 38 → Fail
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 4 (Refactoring – Converting Procedural Code to Functions)

- **Task: Use AI to refactor procedural input–processing logic into functions.**

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

- Expected Output:o Modular code using functions like get_input(), calculate_square(), and display_result().

CODE:

```python
# Refactor the procedural Python script by separating input, processing, and output logic into modular
# functions get_input(), calculate_square(), and display_result() without changing the program behavio
def get_input():
    """
    Get an integer input from the user.

    Returns:
    int: The number entered by the user.
    """
    return int(input("Enter number: "))


def calculate_square(number):
    """
    Calculate the square of a given number.

    Parameters:
    number (int): The number to be squared.

    Returns:
    int: The square of the number.
    """
    return number * number


def display_result(result):
    """
    Display the square result.

    Parameters:
    result (int): The calculated square value.
    """
    print("Square:", result)

# Main Execution Flow
num = get_input()
square = calculate_square(num)
display_result(square)
```

output:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  c:; cd 'c:\Users\perum\OneDrive\Desktop\AI_ASS_CODING'; & 'C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe' '
c:\Users\perum\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57745' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcodi
ng\lab_13.1'
Enter number: 1567
Square: 2455489
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 5 (Refactoring Procedural Code into OOP Design)

- **Task: Use AI to refactor procedural code into a class-based design.Focus Areas:**o Object-Oriented principleso Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

CODE:

```python
# Refactor the procedural salary and tax calculation script into an object-oriented design
# using a class EmployeeSalaryCalculator that applies e
# ncapsulation with attributes and methods while preserving the original functionality
class EmployeeSalaryCalculator:
    """
    A class to calculate employee net salary after tax deduction.

    Attributes:
    salary (float): The gross salary of the employee.
    tax_rate (float): The tax rate applied to the salary.
    """
    def __init__(self, salary, tax_rate=0.2):
        """
        Initialize the EmployeeSalaryCalculator with salary and tax rate.
        Parameters:
        salary (float): The gross salary of the employee.
        tax_rate (float, optional): The tax rate (default is 20%).
        """
        self.salary = salary
        self.tax_rate = tax_rate
    def calculate_tax(self):
        """
        Calculate the tax amount.

        Returns:
        float: Calculated tax amount.
        """
        return self.salary * self.tax_rate
    def calculate_net_salary(self):
        """
        Calculate the net salary after deducting tax.

        Returns:
        float: Net salary after tax deduction.
        """
        tax = self.calculate_tax()
        return self.salary - tax
# Create object
employee = EmployeeSalaryCalculator(50000)
# Print net salary
print(employee.calculate_net_salary())
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  c:; cd 'c:\Users\perum\OneDrive\Desktop\AI_ASS_CODING'; & 'C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe' '
c:\Users\perum\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62646' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcodi
ng\lab_13.1'
40000.0
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 6 (Optimizing Search Logic)

- **Task: Refactor inefficient linear searches using appropriate data structures.**
- Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification

CODE:

```
'''
# Refactor the inefficient linear username search to use an optimized data structure like a set or
# dictionary to achieve O(1) average time complexity with proper justification
def is_username_taken(username, existing_usernames):
    """
    Checks if the given username is already taken.
    """
    # Convert the list of existing usernames to a set for O(1) average time complexity
    username_set = set(existing_usernames)
    return username in username_set
# Example usage
existing_usernames = ["user1", "user2", "user3"]
print(is_username_taken("user2", existing_usernames))  # Output: True
print(is_username_taken("user4", existing_usernames))  # Output: False
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  c:; cd 'c:\Users\perum\OneDrive\Desktop\AI_ASS_CODING'; & 'C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\per
um\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64467' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1'
True
False
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

o add_book(title, author, isbn)

o remove_book(isbn)

o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)

# This code needs refactoring into a proper module with documentation.

library_db = {}

# Adding first book

title = "Python Basics"
```

```
author = "John Doe"
isbn = "101"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
```

```
else:
print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
```

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
a=b
b=c
```

CODE:

```python
# Refactor the procedural Fibonacci code into a modular reusable function generate_fibonacci(n) with
# proper docstrings and test cases while improving structure and efficiency.
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n terms.

    Parameters:
    n (int): Number of terms to generate.

    Returns:
    list: A list containing the Fibonacci sequence up to n terms.
    """
    if n <= 0:
        return []

    if n == 1:
        return [0]

    sequence = [0, 1]

    for _ in range(2, n):
        next_value = sequence[-1] + sequence[-2]
        sequence.append(next_value)

    return sequence


# Test Cases
print("Test Case 1 (n=1):", generate_fibonacci(1))
print("Test Case 2 (n=5):", generate_fibonacci(5))
print("Test Case 3 (n=10):", generate_fibonacci(10))
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  c:; cd 'c:\Users\perum\OneDrive\Desktop\AI_ASS_CODING'; & 'C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\perum\.vs
um\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63816' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1'
Test Case 1 (n=1): [0]
Test Case 2 (n=5): [0, 1, 1, 2, 3]
Test Case 3 (n=10): [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")
```

CODE:

```python
# Refactor the procedural Fibonacci code into a modular reusable function generate_fibonacci(n) with
# proper docstrings and test cases while improving structure and efficiency.
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n terms.

    Parameters:
    n (int): Number of terms to generate.

    Returns:
    list: A list containing the Fibonacci sequence up to n terms.
    """
    if n <= 0:
        return []

    if n == 1:
        return [0]

    sequence = [0, 1]

    for _ in range(2, n):
        next_value = sequence[-1] + sequence[-2]
        sequence.append(next_value)

    return sequence


# Test Cases
print("Test Case 1 (n=1):", generate_fibonacci(1))
print("Test Case 2 (n=5):", generate_fibonacci(5))
print("Test Case 3 (n=10):", generate_fibonacci(10))
```

OUTPUT:

```
um\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63816' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1'
Test Case 1 (n=1): [0]
um\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63816' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1'
Test Case 1 (n=1): [0]
Test Case 1 (n=1): [0]
Test Case 2 (n=5): [0, 1, 1, 2, 3]
Test Case 3 (n=10): [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

 # Chinese Zodiac Program (Unstructured Version)

 # This code needs refactoring.

year = int(input("Enter a year: "))

if year % 12 == 0:

print("Monkey")

elif year % 12 == 1:

print("Rooster")

```
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")
elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.

6. Ensure output remains correct.

CODE:

```python
# Refactor the procedural Chinese Zodiac program into a modular implementation using a reusable function
# get_zodiac(year), replace the if-elif chain with a cleaner data structure, add docstrings, and separate input handling from logic.
def get_zodiac(year):
    """
    Determine the Chinese Zodiac sign for a given year.

    Parameters:
    year (int): The year for which the zodiac sign is required.

    Returns:
    str: The corresponding Chinese Zodiac sign.
    """
    zodiac_cycle = [
        "Monkey", "Rooster", "Dog", "Pig",
        "Rat", "Ox", "Tiger", "Rabbit",
        "Dragon", "Snake", "Horse", "Goat"
    ]

    return zodiac_cycle[year % 12]


def main():
    """
    Handle user input and display the zodiac sign.
    """
    year = int(input("Enter a year: "))
    sign = get_zodiac(year)
    print(sign)


# Program Execution
if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING> cd; cd "c:\Users\perum\OneDrive\Desktop\AI_ASS_CODING"; & "C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe" "c:\Users\perum\.vscode\extensions\ms-ph
hon.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" "53364" "--" "C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1"
Enter a year: 2005
Rooster
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.**For example:**

- **18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ✅ (Harshad Number)**

- **19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ❌ (Not Harshad)**

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10
if sum_digits != 0:
if num % sum_digits == 0:
print("True")
else:
print("False")
else:
print("False")
```

You must:

1. Create a reusable function: is_harshad(number)

2. The function must:

- Accept an integer parameter.
- Return True if the number is divisible by the sum of its digits.
- Return False otherwise.

3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

CODE:

```python
# Refactor the procedural Harshad number checker into a clean reusable function is_harshad(number) with proper docstrings,
# separation of input and logic, boolean return values, and edge-case handling
def is_harshad(number):
    """
    Determine whether a given number is a Harshad (Niven) number.

    A Harshad number is divisible by the sum of its digits.

    Parameters:
    number (int): The number to check.

    Returns:
    bool: True if the number is a Harshad number, otherwise False.
    """
    # Handle negative numbers
    number = abs(number)

    # Edge case: 0 is not considered a Harshad number
    if number == 0:
        return False
    # Calculate sum of digits
    digit_sum = sum(int(digit) for digit in str(number))

    # Avoid division by zero
    if digit_sum == 0:
        return False
    return number % digit_sum == 0
def main():
    """
    Handle user input and display whether the number is Harshad or not.
    """
    num = int(input("Enter a number: "))
    print(is_harshad(num))
if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  & 'C:\Users\perum\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\perum\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\lau
ncher' '53151' '--' 'C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING\ai_assistedcoding\lab_13.1'
Enter a number: 1567
False
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

 # Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
```

count = count + 1

fact = fact // 10

print("Trailing zeros:", count)

You must:

1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
   - Accept a non-negative integer n.
   - Return the number of trailing zeros in **n!**.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

CODE:

```python
# Refactor the factorial trailing zeros program into an optimized reusable function count_trailing_zeros(n)
# using the mathematical approach of counting multiples of 5 without computing the full factorial.
def count_trailing_zeros(n):
    """
    Calculate the number of trailing zeros in n! (factorial of n)
    without computing the full factorial.

    The number of trailing zeros is determined by counting
    the number of factors of 5 in n!.

    Parameters:
    n (int): A non-negative integer.

    Returns:
    int: Number of trailing zeros in n!.
         Returns 0 for n = 0.
         Returns None for negative input.
    """
    if n < 0:
        return None  # Invalid input
    count = 0
    divisor = 5

    while n // divisor > 0:
        count += n // divisor
        divisor *= 5

    return count
def main():
    """
    Handle user input and display trailing zeros result.
    """
    n = int(input("Enter a number: "))
    result = count_trailing_zeros(n)
    if result is None:
        print("Invalid input. Please enter a non-negative number.")
    else:
        print("Trailing zeros:", result)
if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>  c:; cd 'c:\Users\perum\OneDrive\Desktop\AI_ASS_CODIN
hon.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56818' '--' 'C:\Users\perum\OneDrive\Des
Enter a number: 1567
Enter a number: 1567
Trailing zeros: 389
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>




Trailing zeros: 389
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>


Trailing zeros: 389
Trailing zeros: 389
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>


Trailing zeros: 389
PS C:\Users\perum\OneDrive\Desktop\AI_ASS_CODING>
```